HoME: Hash-or-Merge Execution with Multi-Level Adaptation for GPU SpGEMM

Yizhou Xue[2,*], Tengfei Xia[1,2,*], Zhihua Fan[1,✉], Xiaochun Ye[1,2], Wenming Li[1,2],
[1]State Key Lab of Processors, Institute of Computing Technology, CAS, Beijing, China
[2]School of Computer Science and Technology, UCAS, Beijing, China
xueyizhou23@mails.ucas.ac.cn, {xiatengfei24s, fanzhihua, yexiaochun, liwenming}@ict.ac.cn

***Abstract*—Sparse general matrix–matrix multiplication (SpGEMM) exhibits highly irregular execution behavior on GPUs, and neither merge- nor hash-based accumulation performs consistently well across different sparsity structures. We present HoME, a Hash-or-Merge engine with multi-level adaptation for GPU SpGEMM. HoME uses lightweight structural features to select the appropriate accumulation path before GPU execution. For the merge path, it partitions output rows into disjoint column domains, enabling parallel warp-level merging while directly producing sorted CSR rows without global sorting. For the hash path, HoME employs a partitioned-MinHash estimator to approximate per-row output sizes, reducing the cost of exact symbolic computation while retaining an overflow fallback. For AA^T, HoME exploits output symmetry to compute only the upper triangle and reshapes the resulting workloads by splitting heavy rows and fusing light rows. Together, these techniques adapt accumulation, memory provisioning, and task mapping to the input sparsity structure.**

***Index Terms*—SpGEMM, GPU, Adaptive Execution**

I. Introduction

Sparse general matrix–matrix multiplication (SpGEMM) is a core kernel in graph analytics, algebraic multigrid, scientific computing, and machine learning [1–3]. Achieving consistent GPU performance remains difficult because input sparsity determines both the number and distribution of intermediate products and the sparsity pattern of the output. As a result, computational work, memory demand, and access locality vary substantially across matrices and output rows.

The row-wise formulation of Gustavson's algorithm is one of the most widely used dataflows for high-performance SpGEMM because it maps naturally to CSR storage and allows output rows to be computed independently [1, 3, 4]. For each output row $C_{i,:}$, every nonzero $A_{i,k}$ selects the corresponding row $B_{k,:}$ and generates intermediate products. Producing the final row requires these products to be reduced by column index. Multiway merge and hashing are two widely used sparse accumulation mechanisms for this reduction.

Although merge and hash produce the same numerical result, they expose different execution characteristics. Merge preserves column order and directly produces sorted CSR rows, whereas hashing provides concurrent insertion at the cost of table management and output reordering. These costs respond differently to matrix scale, row-length variation, and intermediate-product count. Consistent with these differences, our profiling shows that neither mechanism consistently dominates and that selecting the slower path can increase execution time by up to 26×.

The strong input dependence of accumulator performance calls for adaptation rather than a fixed execution path. Prior GPU SpGEMM systems optimize individual stages of the row-wise workflow. spECK uses lightweight analysis to select accumulator configurations [1], HSMU-SpGEMM improves shared-memory utilization for hash accumulation [4], and Ocean reduces output-sizing overhead through probabilistic estimation [3]. Merge-oriented methods improve ordered reduction through chunking or partitioning. However, these techniques do not jointly select between ordered merge and unordered hash while addressing the dominant bottleneck of the selected path.

These limitations require both input-aware workflow selection and path-specific optimization. To meet these requirements, we present HoME, a Hash-or-Merge Engine for row-wise GPU SpGEMM. HoME first uses lightweight structural features to select the accumulation path. When merge is selected, column-domain partitioning exposes intra-row parallelism while preserving globally ordered output. When hash is selected, Light-Symbolic MinHash estimates output-row sizes without exact symbolic enumeration. The resulting workflow selects an accumulator for the input structure and then targets the principal cost of that accumulator.

The main contributions of this work are as follows:

- We characterize the complementary performance regions of merge and hash under a common row-wise dataflow. Based on this characterization, we develop a lightweight dispatcher using matrix scale, row imbalance, and estimated intermediate-product count.
- We develop column-domain parallel merge, which partitions a heavy output row into independently ordered column ranges. The resulting ranges can be processed concurrently and concatenated directly into a sorted CSR row.
- We develop Light-Symbolic MinHash, which replaces exact output sizing with compact structural summaries. An overflow-safe fallback preserves correctness when the estimated capacity is insufficient.

✉ Corresponding author. * Equal contribution.

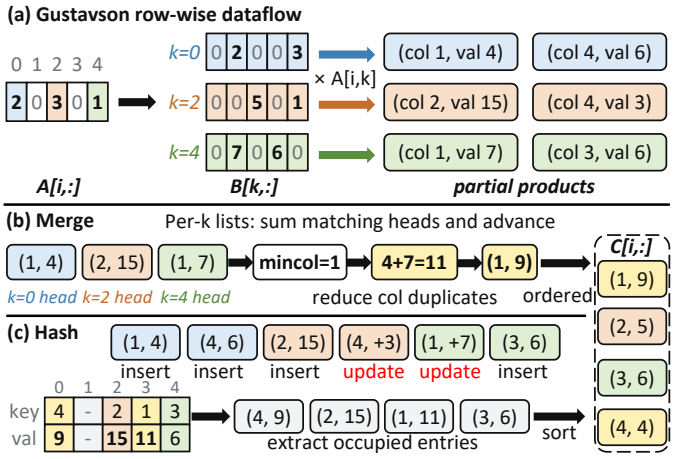

Fig. 1: Merge- and Hash-based Gustavson SpGEMM.

II. Background and Motivation

A. Row-Wise Gustavson SpGEMM

Given sparse matrices $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$, SpGEMM computes $C = AB$. For an output row $C_{i,:}$, let $\mathcal{K}_i = \{k \mid A_{i,k} \neq 0\}$. The row-wise formulation of Gustavson's algorithm computes

$$C_{i,:} = \sum_{k\in\mathcal{K}_i} A_{i,k}B_{k,:}. \tag{1}$$

Equation 1 expresses each output row as a reduction over the rows of B referenced by $A_{i,:}$. With column-sorted CSR storage, the nonzeros in $A_{i,:}$ and every referenced row $B_{k,:}$ can be traversed contiguously. Since different output rows share no output locations, they can be assigned to independent GPU thread groups. Gustavson's formulation therefore combines efficient sparse traversal with parallelism across output rows.

Each scaled row $A_{i,k}B_{k,:}$ in Equation 1 produces a column-ordered list of intermediate products. The remaining operation is to combine products that contribute to the same output column. A merge accumulator repeatedly selects the smallest leading column and combines equal columns, which directly produces a sorted output row. A hash accumulator instead uses the output column as a key and combines products mapped to the same key. Its occupied entries are subsequently extracted and sorted. Figure 1 illustrates how the two accumulators reduce the same intermediate products.

HoME keeps the row traversal and output-row ownership fixed across both paths and adapts only the accumulation mechanism. Holding the underlying dataflow constant isolates the performance effects of the accumulator. It also exposes two related design questions: which accumulator should process a given input, and how should the selected path address its dominant bottleneck.

B. Motivation

Accumulator complementarity. To answer the first question, we profile merge and hash under the common dataflow described above. The winning path changes with matrix scale, row imbalance, and intermediate-product count. Neither accumulator consistently dominates, and selecting the slower path can increase execution time by up to 26×. This performance variation motivates a lightweight dispatcher that selects the accumulation path from input structure.

Merge critical path. Path selection alone does not remove the internal bottleneck of merge. A row-wise multiway merge advances one or more input lists after each minimum-column selection, and the next selection depends on the updated list positions. This loop-carried dependency limits progress within an output row. When intermediate-product counts are highly skewed, a small number of heavy rows can therefore determine kernel completion time, while parallelism across output rows cannot shorten their critical paths.

Hash sizing overhead. The hash path has a different bottleneck because accumulator capacity must be determined before numerical execution. Exact symbolic computation obtains this capacity by enumerating and deduplicating intermediate columns. Numerical accumulation subsequently processes the same intermediate products again, so the symbolic work scales with intermediate-product count rather than only with input size. This repeated processing can account for a substantial fraction of end-to-end runtime and motivates a lower-cost sizing mechanism for the selected hash path.

III. Our Design

A. Adaptive Workflow Dispatch

B. Column-Domain Parallel Merge

For an output row $C_{i,:}$, every nonzero $A_{i,k}$ introduces a sorted sequence from $B_{k,:}$. Let $\mathcal{K}_i = \{k \mid A_{i,k} \neq 0\}$. The row computation can be expressed as

$$C_{i,:} = \sum_{k\in\mathcal{K}_i} A_{i,k}B_{k,:}. \tag{2}$$

A conventional row-wise merge repeatedly selects the smallest column index among these sequences and combines equal indices. Each selection changes the active position of one or more sequences, which creates a loop-carried dependency. A heavy output row therefore has limited intra-row parallelism and can dominate the kernel completion time.

HoME introduces column-domain parallelism to shorten this critical path. We divide the output column space into P ordered and disjoint domains $\mathcal{D}_0, \ldots, \mathcal{D}_{P-1}$. Each sorted input sequence is partitioned according to the same boundaries. HoME then assigns each pair (i, p), representing output row i and domain $\mathcal{D}_p$, to an independent warp. Within a domain, the warp uses shuffle operations to exchange candidate elements, select the smallest column index, and accumulate contributions with identical indices. Different domains access disjoint output columns and can execute concurrently without cross-domain conflicts.

Each warp produces a locally sorted sequence. Since every column in $\mathcal{D}_p$ precedes every column in $\mathcal{D}_{p+1}$, the domain results are concatenated in domain order to form $C_{i,:}$. This directly produces a column-sorted CSR row and removes the

need for a global sorting pass. The aggregate merge work remains proportional to the number of intermediate products. The additional cost is limited to domain partitioning and synchronization, while long rows gain an extra axis of parallel execution.

C. Light-Symbolic Output Sizing

Hash accumulation requires the capacity of each output row before numerical execution. For $\mathcal{K}_i = \{k \mid A_{i,k} \neq 0\}$, the structural size of an output row is

$$\mathrm{nnz}(C_{i,:}) = \left| \bigcup_{k \in \mathcal{K}_i} \{j \mid B_{k,j} \neq 0\} \right|. \tag{3}$$

Exact symbolic execution enumerates the intermediate products and removes duplicate column indices. Its work is therefore proportional to the multiplication FLOPs and partially repeats the traversal and lookup operations of numerical accumulation.

HoME replaces exact counting with a compact partitioned MinHash sketch. Each column index is mapped to a 32-bit value using MurmurHash. The low seven bits select one of 128 sketch registers, and the minimum hash value observed by each register is retained using `atomicMin`. For every row $B_{k,:}$, this process constructs a fixed-size structural summary S_k. The sketch construction scans the input nonzeros once and exposes parallel updates across both rows and registers.

MinHash sketches support inexpensive union. To estimate the structure of $C_{i,:}$, HoME combines the sketches of all rows selected by $\mathcal{K}_i$ and computes

$$S_i^C[b] = \min_{k \in \mathcal{K}_i} S_k[b], \qquad 0 \leq b < 128. \tag{4}$$

The resulting register minima are converted into an estimate $\widehat{n}_i$ of $\mathrm{nnz}(C_{i,:})$ using MinHash order statistics. Since the sketch size is fixed, the sizing stage scales with the number of input nonzeros rather than the number of intermediate products.

HoME allocates each hash accumulator with a capacity of $1.5\widehat{n}_i$. This factor absorbs typical estimation error while limiting temporary memory and post-processing cost. If a row still exceeds its assigned capacity, the device records an overflow flag and HoME falls back to the merge path. The estimate only controls resource provisioning and does not alter numerical accumulation, while the fallback prevents an underestimated row from producing an incomplete result.

IV. Evaluation

A. Experiment Setup

Methodology. We conduct all experiments on a single NVIDIA H100 PCIe GPU using CUDA 12.8. All methods use 32-bit indices and FP64 values. Each matrix–method pair is executed for 10 warm-up runs followed by 10 measured runs, and we report the arithmetic mean of the measured runtimes. The reported time excludes file I/O and host–device data transfers, but includes all method-specific online analysis, symbolic or estimation phases, memory allocation, numerical accumulation, and required post-processing.

Baselines. We compare HoME against NVIDIA cuSPARSE and five state-of-the-art GPU SpGEMM implementations: spECK [1], opSparse [2], TileSpGEMM [5], HSMU-SpGEMM [4], and Ocean [3]. We use their publicly available implementations and make only minor compatibility modifications required to run them on our H100 platform, without changing their core algorithms or optimization strategies. We verify the correctness of HoME and all baseline implementations before collecting performance results.

Benchmarks. We adopt the same two benchmark datasets as Ocean, comprising 337 square matrices for evaluating $C = AA$ and 64 rectangular matrices for evaluating $C = AA^T$ from the SuiteSparse Matrix Collection [6]. These matrices span diverse matrix sizes, sparsity levels, and sparsity structures, and originate from a broad range of real-world applications.

B. Experiment results

V. Conclusion

VI. Acknowledgment

References

[1] M. Parger, M. Winter, D. Mlakar, and M. Steinberger, "speck: accelerating gpu sparse matrix-matrix multiplication through lightweight analysis," in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '20, (New York, NY, USA), p. 362–375, Association for Computing Machinery, 2020.

[2] Z. Du, Y. Guan, T. Guan, D. Niu, L. Huang, H. Zheng, and Y. Xie, "Opsparse: A highly optimized framework for sparse general matrix multiplication on gpus," *IEEE Access*, vol. 10, pp. 85960–85974, 2022.

[3] Y. Li and G. Guidi, "Ocean: Fast estimation-based sparse general matrix-matrix multiplication on gpu," in *Proceedings of the 2026 International Conference on Supercomputing (ICS '26)*, (Belfast, United Kingdom), pp. –, ACM, July 2026.

[4] M. Wu, H. Luo, F. Li, Y. Zhang, Z. Tang, K. Li, J. Zhang, and C. Liu, "Hsmu-spgemm: Achieving high shared memory utilization for parallel sparse general matrix-matrix multiplication on modern gpus," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1452–1466, 2025.

[5] Y. Niu, Z. Lu, H. Ji, S. Song, Z. Jin, and W. Liu, "Tilespgemm: a tiled algorithm for parallel sparse general matrix-matrix multiplication on gpus," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '22, (New York, NY, USA), p. 90–106, Association for Computing Machinery, 2022.

[6] S. P. Kolodziej, M. Aznaveh, M. Bullock, J. David, T. A. Davis, M. Henderson, Y. Hu, and R. Sandstrom, "The suitesparse matrix collection website interface," *Journal of Open Source Software*, vol. 4, no. 35, p. 1244, 2019.